LogicPuzzleX: One Constraint Engine for Sudoku and Kakuro

Md Shihabul Alam Shihab, Rayed Riasat Rabbi, Rifat Imtiaze Ruddro, Sayma Akhter Snigdha
Department of ECE, North South University

I. Introduction

Logic puzzles such as Sudoku and Kakuro are constraint satisfaction problems (CSPs): assign a value to every variable so that every constraint holds [1]. Generalized Sudoku is NP-complete [2], yet good CSP techniques solve typical 9×9 puzzles in milliseconds. LogicPuzzleX models each puzzle as a factor graph, solves it with a ladder of increasingly capable algorithms and shows the search live in the browser. We measure what each technique contributes and compare CSP search with propositional logic on the same task. Four questions drive the work:

RQ1: How much does each technique cut search effort and runtime? **RQ2:** Does solver effort predict rated difficulty, and how many puzzles need no search? **RQ3:** How do binary, global and CNF formulations compare? **RQ4:** How does one engine scale from 4×4 to 16×16 Sudoku, and to Kakuro?

II. Approach

A puzzle is a factor graph: variables X_i with domains, and factors $f_j(x) \in \{0, 1\}$. A complete assignment x solves it when

$$\text{weight}(x) = \prod_{j=1}^{m} f_j(x) = 1. \tag{1}$$

A 9×9 Sudoku has 81 variables and 810 binary $\neq$ factors, or 27 global AllDifferent factors. Givens are fixed by conditioning, and a cell's 20 peers form its Markov blanket. Search extends a partial assignment x with $X_i = v$ only if every dependent factor in $D(x, X_i)$ stays satisfied [3]:

$$\delta(x, X_i, v) = \prod_{f \in D(x, X_i)} f\big(x \cup \{X_i : v\}\big) = 1. \tag{2}$$

One recursive search serves the whole ladder in Table I: each rung changes only the variable order, the value order or the inference step. The logic track encodes the same puzzle in CNF [4] with 729 symbols and 11,745 clauses, and solves it with DPLL [5]. A solution s is unique exactly when $\text{KB} \cup \{\neg s\}$ is unsatisfiable, that is, when $\text{KB} \models s$. Fig. 1 shows how the parts connect: every solver emits the same event stream, and that stream drives every animation in the app.

III. Progress

All of Sudoku already works end to end, ahead of the minimal version planned for this update. All seven algorithms run on 4×4, 6×6, 9×9 and 16×16 grids, and 166 automated tests pass. The generator produces puzzles of a requested grade with exactly one solution, and Explain mode turns solver events into plain-English deductions. In the browser app (Fig. 2) a user can play with live solvability checks and hints, replay the search, race up to four algorithms and browse the benchmarks. Kakuro is the main work left: its sum factors, propagation and board, which RQ4 needs. The demo video, a README GIF and the final report also remain.

TABLE I
The algorithm ladder

ID	Variable order	Value order	Inference
`bt`	static	ascending	none
`fc`	static	ascending	forward checking [6]
`mrv`	MRV, degree	ascending	forward checking
`lcv`	MRV, degree	LCV	forward checking
`mac`	MRV, degree	LCV	AC-3 [7]
`mac+`	MRV, degree	LCV	AC-3, hidden singles [8]
`dpll`	shortest clause	true first	unit propagation [5]

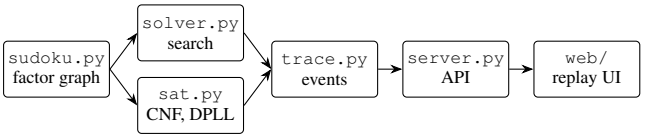


Fig. 1. Architecture. Both solver families feed one event stream.

IV. Preliminary Results

We ran each algorithm on 1000 puzzles, 250 per difficulty bucket, sampled from a rated dataset of 3 million [9], with a 10 s limit per run and the median of 3 timed runs on a 4-vCPU Kaggle session (Fig. 3). MRV gives the largest single gain, 23× fewer nodes than forward checking (paired median), while LCV adds nothing here (Table II). MAC roughly halves the nodes again yet runs slower than MRV (median 44 ms against 27 ms). Hidden singles give the fastest solver: `mac+` takes a median 24 ms and needs no backtracking on any puzzle in the easy bucket.

Naive backtracking times out on 33% of puzzles, and at similar rates in every bucket (30–36%), so its failures barely follow difficulty. Rank correlation between rating and nodes peaks at $\rho = 0.475$ for `mac+`; the rating comes from an automated solver rather than from people, so this is not direct evidence about human difficulty. Global factors halve MAC's time without changing its search, and 243 redundant clauses cut DPLL's nodes sixfold. On 8 generated 16×16 puzzles only

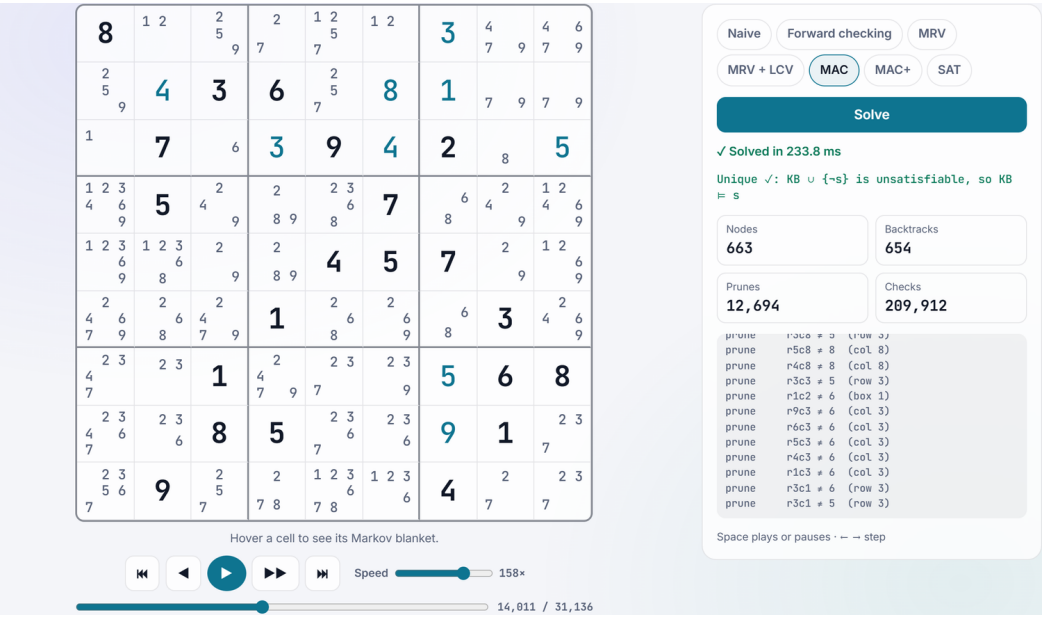


Fig. 2. "Watch it think" part-way through a replay of `mac` on Inkala's 2012 puzzle, with the SAT uniqueness proof above the statistics.

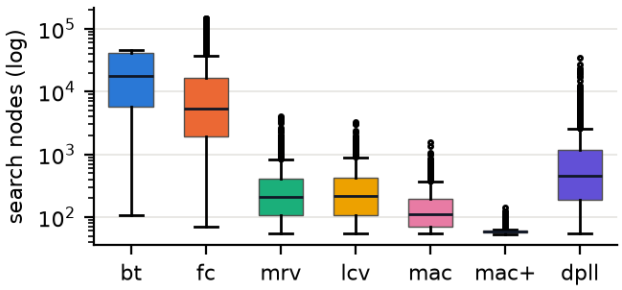


Fig. 3. Search nodes per algorithm on 1000 rated puzzles. A run that hits the 10 s limit counts the nodes it reached, so `bt` and `fc` are lower bounds.

TABLE II
MEDIAN EFFORT ON 1000 RATED PUZZLES

ID	Nodes	Time (ms)	Timed out
`bt`	17,890	4,185	32.9%
`fc`	5,446	406	2.3%
`mrv`	210	27	0%
`lcv`	219	30	0%
`mac`	113	44	0%
`mac+`	58	24	0%
`dpll`	448	103	0%

`mac+` finished (median 208 ms); the other six hit the limit on all 8.

V. Plan to Final

3–16 Nov: Kakuro on the same engine, with sum factors and a board in the app. **17–30 Nov:** Kakuro in the scaling experiment, README GIF, feature freeze. **December:** 8-page report, final slides, one-minute video and rehearsals.

TABLE III
COURSE CONCEPT MAP

Feature	Course concept	Vol.
Puzzle to variables and factors	Factor graphs, CSP	3
Givens fixed before search	Conditioning	3
A cell and its 20 peers	Markov blanket	3
Search over partial assignments	Backtracking, DFS	1
Forward checking, AC-3	Constraint propagation	3
Sudoku to CNF, DPLL	Propositional logic	4
Uniqueness proof	Entailment	4
Explain mode	Inference rules	4

AI use: Claude Code [10] assisted with planning, code, experiments and drafting this report.

References

[1] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson, 2020.

[2] T. Yato and T. Seta, "Complexity and completeness of finding another solution and its application to puzzles," *IEICE Trans. Fundamentals*, vol. E86-A, no. 5, 2003.

[3] CSE 440 course notes, Vols. 1–4, North South University, 2026.

[4] I. Lynce and J. Ouaknine, "Sudoku as a SAT problem," in *Proc. ISAIM*, 2006.

[5] M. Davis, G. Logemann, and D. Loveland, "A machine program for theorem-proving," *Commun. ACM*, vol. 5, no. 7, pp. 394–397, 1962.

[6] R. M. Haralick and G. L. Elliott, "Increasing tree search efficiency for constraint satisfaction problems," *Artif. Intell.*, vol. 14, no. 3, pp. 263–313, 1980.

[7] A. K. Mackworth, "Consistency in networks of relations," *Artif. Intell.*, vol. 8, no. 1, pp. 99–118, 1977.

[8] H. Simonis, "Sudoku as a constraint problem," in *CP Workshop on Modeling and Reformulating CSPs*, 2005.

[9] D. Radcliffe, "3 million Sudoku puzzles with ratings," Kaggle dataset, CC0-1.0, accessed Oct. 2026. [Online]. Available: https://www.kaggle.com/datasets/radcliffe/3-million-sudoku-puzzles-with-ratings

[10] Anthropic, Claude Code (Claude Opus 5.5), AI assistant used for planning, code, experiments and drafting this report, Oct. 2026.